# Linear Sieve SPF Algorithm

explain me  liner sieve for the obtainting the smallest prime factor in exact response with the algo and intuttion and with the code and dry of this

## Linear Sieve (Euler's Sieve) for Smallest Prime Factor (SPF)

The **Linear Sieve** is one of the most useful number theory algorithms for competitive programming.

It computes:

- All prime numbers up to `N`
- The **Smallest Prime Factor (SPF)** of every number from `2` to `N`

in **O(N)** time.

---

## Why do we need SPF?

Suppose you want to factorize many numbers.

Example:

```
84 = 2 × 2 × 3 × 7
```

Normally you divide by every integer.

Time:

```
O(√N)
```

for one number.

If there are

```
10^5 numbers
```

then this becomes slow.

Instead, precompute

```
spf[x] = smallest prime that divides x
```

Example

| Number | SPF |
|---|---|
| 2 | 2 |
| 3 | 3 |
| 4 | 2 |
| 5 | 5 |
| 6 | 2 |
| 7 | 7 |
| 8 | 2 |
| 9 | 3 |
| 10 | 2 |
| 11 | 11 |
| 12 | 2 |

Now

```
84
↓
spf[84]=2
84/2=42

spf[42]=2
42/2=21

spf[21]=3
21/3=7

spf[7]=7
```

Factorization becomes

```
84 = 2 × 2 × 3 × 7
```

Time:

```
O(number of prime factors)
```

which is extremely fast.

---

## First intuition

Normal Sieve of Eratosthenes marks multiples many times.

Example

```
12

Marked by 2
Marked by 3
```

30

```
Marked by 2
Marked by 3
Marked by 5
```

So one number gets processed repeatedly.

---

## Linear Sieve Idea

We want

> **Every composite number should be generated exactly once.**

That is why its complexity becomes

```
O(N)
```

instead of

```
O(N log log N)
```

## Main Idea

Maintain two things

```
vector<int> primes;
spf[]
```

For every number

```
i = 2...N
```

## Case 1

If

```
spf[i]==0
```

then

```
i is prime
```

because nobody divided it before.

So

```
spf[i]=i
```

and

```
primes.push_back(i)
```

Now use all previous primes to generate new composite numbers.

## The most important observation

Suppose

```
i = 6
```

Current primes

```
2 3 5
```

Generate

```
6×2 =12
6×3 =18
6×5 =30
```

Assign

```
spf[12]=2
spf[18]=3
spf[30]=5
```

But should we continue forever?

No.

There is one important stopping condition.

## The Break Condition

Stop when

```
prime > spf[i]
```

or equivalently

```
C++
if(prime == spf[i]) continue;
```

The usual implementation is

```
C++
if(prime == spf[i]) break;
```

Actually the correct condition is

```
C++
if(prime > spf[i]) break;
```

Most implementations write

```
C++
if(p == spf[i]) break;
```

inside the loop because the primes are increasing, and after reaching `spf[i]`, larger primes would also violate the condition.

The standard implementation is:

```
C++
for (int p : primes)
{
    if (p > spf[i]) break;
    if (i * p > N) break;

    spf[i * p] = p;
}
```

## Why do we stop?

This is the heart of Linear Sieve.

Suppose

```
i=6

spf[6]=2
```

Current primes

```
2 3 5
```

Generate

```
6×2=12
```

Good.

Next

```
6×3=18
```

Should we generate?

No.

Why?

Because

```
18
```

will later be generated as

```
9×2
```

whose smallest prime factor is

```
2
```

So we stop.

Thus every number gets produced only once.

---

## Complete Algorithm

```
Initialize spf[] with 0.

Create empty primes list.

For i from 2 to N

    if spf[i]==0

        spf[i]=i
        primes.push_back(i)

    for every prime p

        if p>spf[i]
              break

        if i*p>N
              break

        spf[i*p]=p
```

## Why does this work?

Suppose

```
x=i×p
```

We only allow

```
p<=spf[i]
```

Therefore

```
p
```

becomes the smallest prime factor of

```
x
```

No other iteration can assign SPF to this number again.

Hence

Every composite is visited

```
Exactly One Time
```

## Dry Run

Let

```
N=20
```

Initially

```
spf[]=0

primes=[]
```

## i=2

```
spf[2]==0

Prime
```

```
spf[2]=2

primes={2}
```

Generate

```
2×2=4

spf[4]=2
```

Current

```
spf

2→2
4→2
```

## i=3

```
spf[3]==0

Prime
```

```
spf[3]=3

primes

2 3
```

Generate

```
3×2=6

spf[6]=2
```

Next

```
3×3=9

spf[9]=3
```

Stop because

```
3==spf[3]
```

Current

```
spf

2→2
3→3
4→2
```

```
6→2
9→3
```

## i=4

Already marked

```
spf[4]=2
```

Generate

```
4×2=8
spf[8]=2
```

Stop because

```
2==spf[4]
```

## i=5

Prime

```
spf[5]=5
```

Primes

```
2 3 5
```

Generate

```
5×2=10
spf[10]=2
```

Generate

```
5×3=15
spf[15]=3
```

Generate

```
5×5=25
Out of range
```

Done.

---

Continue like this.

Finally

| Number | SPF |
|---|---|
| 2 | 2 |
| 3 | 3 |
| 4 | 2 |
| 5 | 5 |
| 6 | 2 |
| 7 | 7 |
| 8 | 2 |
| 9 | 3 |
| 10 | 2 |
| 11 | 11 |
| 12 | 2 |
| 13 | 13 |
| 14 | 2 |
| 15 | 3 |
| 16 | 2 |

| Number | SPF |
|---|---|
| 17 | 17 |
| 18 | 2 |
| 19 | 19 |
| 20 | 2 |

## Complete C++ Code (with inline explanation)

```
C++
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 1000000;

vector<int> primes;     // Stores all prime numbers
vector<int> spf(MAXN + 1, 0); // spf[x] = smallest prime factor of x

void linearSieve(int n)
{
    // Traverse every number from 2 to n
    for (int i = 2; i <= n; i++)
    {
        // If spf is still 0, nobody marked it.
        // Therefore it is a prime.
        if (spf[i] == 0)
        {
            spf[i] = i;
            primes.push_back(i);
        }

        // Generate composite numbers using previously found primes.
        for (int p : primes)
        {
            // Don't go outside the limit.
            if (1LL * i * p > n)
                break;

            // Stop when p becomes larger than the smallest
            // prime factor of i.
            // This guarantees every composite is generated exactly once.
            if (p > spf[i])
                break;

            // First time this composite is generated,
            // so p is its smallest prime factor.
            spf[i * p] = p;
        }
    }
}

int main()
{
    linearSieve(20);

    for (int i = 2; i <= 20; i++)
    {
        cout << "SPF[" << i << "] = " << spf[i] << "\n";
    }
}
```

## Complexity

### Time

Every composite number is marked **exactly once**.

```
O(N)
```

### Space

```
SPF array : O(N)

Prime list : O(N / log N)

Overall : O(N)
```

## Comparison

| Algorithm | Time | Multiple markings? | Computes SPF? |
|---|---|---|---|
| Trial Division | O(√N) per query | No | No |
| Sieve of Eratosthenes | O(N log log N) | Yes | Can be modified to compute SPF |
| **Linear Sieve (Euler Sieve)** | **O(N)** | **No** | **Yes (naturally)** |

## Interview intuition (the one-line takeaway)

> **The Linear Sieve works because every composite number is generated exactly once—from the unique pair `(i, p)` where `p` is the smallest prime factor of that composite. By stopping when `p > spf[i]`, we prevent duplicate generations, giving an overall time complexity of `O(N)`.**

now give me for the sieve of the erothesis with the same format

## Sieve of Eratosthenes

The **Sieve of Eratosthenes** is one of the oldest and most important algorithms in number theory.

It is used to find **all prime numbers up to N** efficiently.

Unlike the Linear Sieve, its main purpose is **finding primes**, although it can also be modified to compute the Smallest Prime Factor (SPF).

Its time complexity is

```
O(N log log N)
```

## Why do we need it?

Suppose you want to find all primes up to

```
N = 1,000,000
```

The naïve way is:

For every number

```
2, 3, 4, 5, ..., N
```

check whether it is prime.

Each primality check costs

```
O(√N)
```

Overall complexity becomes

```
O(N√N)
```

which is far too slow.

Instead, the Sieve computes **all primes together** in nearly linear time.

---

## First Intuition

A prime number has exactly two divisors:

```
1 and itself
```

Every composite number is a multiple of some smaller prime.

For example,

```
12 = 2 × 6
15 = 3 × 5
18 = 2 × 9
20 = 2 × 10
21 = 3 × 7
```

Instead of checking every number individually,

we **remove all multiples of every prime.**

Whatever remains is prime.

---

## Main Idea

Initially assume

```
Every number is prime.
```

Then,

Start from

```
2
```

Since 2 is prime,

mark all its multiples as composite.

Then move to

```
3
```

Since 3 is still unmarked,

it is prime.

Mark all multiples of 3.

Continue this process.

Finally,

only primes remain unmarked.

---

## Visualization

Initially

```
2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
```

All are assumed prime.

---

## Step 1

Current prime

```
2
```

Mark multiples

```
4 6 8 10 12 14 16 18 20
```

Remaining

```
2 3 5 7 9 11 13 15 17 19
```

## Step 2

Current prime

```
3
```

Mark multiples

```
6 9 12 15 18
```

Newly removed

```
9 15
```

Remaining

```
2 3 5 7 11 13 17 19
```

## Step 3

Current prime

```
4
```

Already marked.

Skip.

---

## Step 4

Current prime

```
5
```

Mark multiples

```
10 15 20
```

Already marked.

Nothing changes.

---

Finally

```
2 3 5 7 11 13 17 19
```

These are all primes ≤ 20.

---

## Why do we start marking from $i^2$?

Suppose

```
i = 5
```

Multiples are

```
5×2 = 10
5×3 = 15
5×4 = 20
5×5 = 25
```

Notice

```
10
```

was already marked by

```
2
```

Similarly,

```
15
```

was marked by

```
3
```

and

```
20
```

was marked by

```
2
```

Therefore,

every multiple before

```
i²
```

has already been handled.

So we only begin from

```
i²
```

This avoids unnecessary work.

## Why do we only iterate until √N?

Suppose

```
N = 100
```

If a composite number exists,

it must have a factor ≤ √100 = 10.

For example,

```
77 = 7 × 11
```

Small factor

```
7
```

is already ≤ √100.

So once we process every prime up to √N,

all composites are guaranteed to be marked.

## Complete Algorithm

```
Create an array isPrime[].

Initially mark every number as true.

Set

0 → false
1 → false

For every i from 2 to √N

    If i is prime

        Mark every multiple

        i*i, i*i+i, i*i+2i...

        as composite

Finally,

every remaining true index is prime.
```

## Dry Run

Suppose

```
N = 20
```

Initially

| Number | Prime? |
|---|---|
| 0 | False |
| 1 | False |
| 2 | True |
| 3 | True |
| 4 | True |
| 5 | True |
| 6 | True |
| 7 | True |
| 8 | True |
| 9 | True |
| 10 | True |
| 11 | True |
| 12 | True |
| 13 | True |
| 14 | True |

| Number | Prime? |
|---|---|
| 15 | True |
| 16 | True |
| 17 | True |
| 18 | True |
| 19 | True |
| 20 | True |

## i = 2

Prime

Mark

```
4 6 8 10 12 14 16 18 20
```

## i = 3

Prime

Start from

```
9
```

Mark

```
9 12 15 18
```

## i = 4

Already composite

Skip.

Since

```
5 × 5 = 25 > 20
```

Stop.

Remaining primes

```
2
3
5
7
11
13
17
19
```

## Complete C++ Code (with inline explanation)

```
C++
#include <bits/stdc++.h>
using namespace std;

int main()
{
    int n = 20;

    // Assume every number is prime
    vector<bool> isPrime(n + 1, true);

    // 0 and 1 are not prime
    isPrime[0] = false;
    isPrime[1] = false;

    // Only iterate until sqrt(n)
    for (int i = 2; i * i <= n; i++)
    {
        // If current number is prime
        if (isPrime[i])
```

```
        {
            // Start marking from i*i because
            // smaller multiples have already
            // been marked by smaller primes.
            for (int j = i * i; j <= n; j += i)
            {
                isPrime[j] = false;
            }
        }
    }

    cout << "Prime Numbers:\n";

    for (int i = 2; i <= n; i++)
    {
        if (isPrime[i])
            cout << i << " ";
    }

    cout << endl;
}
```

Output

```
Prime Numbers:
2 3 5 7 11 13 17 19
```

## Modified Sieve to Compute Smallest Prime Factor (SPF)

The normal sieve only tells us whether a number is prime.

To compute the **Smallest Prime Factor (SPF)**, we slightly modify the marking process.

Whenever we mark a multiple for the **first time**, we store the current prime as its SPF.

```
C++
#include <bits/stdc++.h>
using namespace std;

int main()
{
    int n = 20;

    vector<int> spf(n + 1);

    // Initially, every number is its own SPF
    for (int i = 0; i <= n; i++)
        spf[i] = i;

    // Process all primes
    for (int i = 2; i * i <= n; i++)
    {
        // i is prime if it has not been changed
        if (spf[i] == i)
        {
            // Mark multiples starting from i*i
            for (int j = i * i; j <= n; j += i)
            {
                // Assign SPF only once
                if (spf[j] == j)
                    spf[j] = i;
            }
        }
    }

    for (int i = 2; i <= n; i++)
    {
        cout << "SPF[" << i << "] = " << spf[i] << "\n";
    }
}
```

Output

```
SPF[2] = 2
SPF[3] = 3
SPF[4] = 2
SPF[5] = 5
SPF[6] = 2
SPF[7] = 7
SPF[8] = 2
SPF[9] = 3
SPF[10] = 2
SPF[11] = 11
SPF[12] = 2
SPF[13] = 13
SPF[14] = 2
SPF[15] = 3
SPF[16] = 2
SPF[17] = 17
SPF[18] = 2
SPF[19] = 19
SPF[20] = 2
```

## Complexity

### Time

Outer loop

```
O(√N)
```

Inner loop marks multiples.

Overall complexity is

```
O(N log log N)
```

This is a proven result based on the harmonic sum over primes.

---

### Space

```
isPrime[] : O(N)
or
SPF[] : O(N)
```

Overall

```
O(N)
```

## Comparison: Sieve of Eratosthenes vs Linear Sieve

| Feature | Sieve of Eratosthenes | Linear Sieve (Euler Sieve) |
|---|---|---|
| Time Complexity | **O(N log log N)** | **O(N)** |
| Space Complexity | O(N) | O(N) |
| Number of markings | A composite may be marked many times | Every composite is generated exactly once |
| Prime List | Yes | Yes |
| SPF Generation | Requires a small modification | Built in naturally |
| Implementation | Simpler | Slightly more complex |
| Best for | Finding all primes up to N | Finding both primes and SPF efficiently |

---

## Interview intuition (the one-line takeaway)

> **The Sieve of Eratosthenes works by repeatedly taking the next unmarked number (which must be prime) and marking all of its multiples as composite. By starting from $i^2$, it avoids redundant work because all smaller multiples have already been marked by smaller primes, achieving a time complexity of $O(N \log \log N)$.**

now give me the code , where check is number of prime or not using the linear  sieve and if prime the find the prime factor of it

There is one small correction in your question:

> **If a number is prime, its prime factorization is just the number itself.**

For example,

```
13 → Prime
Prime Factors = 13
```

Usually, what we want is:

1. Build the **Linear Sieve**.
2. Check if a number is prime.
3. If it is **not prime**, find its prime factorization using the `spf[]` array.
4. If it **is prime**, simply print that the prime factor is the number itself.

---

## Complete Code

```
C++
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 1000000;

vector<int> spf(MAXN + 1);
```

```
vector<int> primes;

// Linear Sieve
void linearSieve(int n)
{
    for (int i = 2; i <= n; i++)
    {
        // If not marked, it is prime
        if (spf[i] == 0)
        {
            spf[i] = i;
            primes.push_back(i);
        }

        // Generate composites exactly once
        for (int p : primes)
        {
            if (1LL * i * p > n)
                break;

            if (p > spf[i])
                break;

            spf[i * p] = p;
        }
    }
}

// O(1) Prime Check
bool isPrime(int x)
{
    return x >= 2 && spf[x] == x;
}

// O(log N) Prime Factorization
vector<int> primeFactorization(int x)
{
    vector<int> factors;

    while (x > 1)
    {
        factors.push_back(spf[x]);
        x /= spf[x];
    }

    return factors;
}

int main()
{
    linearSieve(MAXN);

    int n;
    cin >> n;

    if (isPrime(n))
    {
        cout << n << " is Prime\n";
        cout << "Prime Factors : " << n << '\n';
    }
    else
    {
        cout << n << " is Not Prime\n";

        vector<int> factors = primeFactorization(n);

        cout << "Prime Factors : ";

        for (int x : factors)
            cout << x << " ";

        cout << '\n';
    }

    return 0;
}
```

## Dry Run 1

Input

```
84
```

Prime check

```
spf[84] = 2

84 != 2

Therefore

84 is NOT Prime
```

Factorization

```
84

spf[84] = 2

84 / 2 = 42
```

```
Factors
2
```

```
42
spf[42] = 2
42 / 2 = 21
Factors
2 2
```

```
21
spf[21] = 3
21 / 3 = 7
Factors
2 2 3
```

```
7
spf[7] = 7
7 / 7 = 1
Factors
2 2 3 7
```

Output

```
84 is Not Prime
Prime Factors : 2 2 3 7
```

## Dry Run 2

Input

```
97
```

Prime check

```
spf[97] = 97
97 == spf[97]
```

Therefore

```
97 is Prime
Prime Factors : 97
```

## Time Complexity

### Preprocessing (Linear Sieve)

```
O(N)
```

Done only once.

---

### Prime Check

```
C++
isPrime(x)
```

Only one comparison

```
C++
spf[x] == x
```

Time

```
O(1)
```

### Prime Factorization

Every iteration divides the number by one of its prime factors.

```
C++
while (x > 1)
{
    x /= spf[x];
}
```

Time

```
O(number of prime factors)
```

Worst case

```
O(log N)
```

## A More Useful Version (Prime Factors with Their Powers)

In competitive programming, you often need the factorization in the form:

```
360 = 2³ × 3² × 5¹
```

You can compute it like this:

```
C++
vector<pair<int,int>> factorize(int x)
{
    vector<pair<int,int>> ans;

    while (x > 1)
    {
        int p = spf[x];
        int cnt = 0;

        while (x % p == 0)
        {
            cnt++;
            x /= p;
        }

        ans.push_back({p, cnt});
    }

    return ans;
}
```

Example:

```
C++
int n = 360;

auto factors = factorize(n);

for (auto [prime, power] : factors)
{
    cout << prime << "^" << power << " ";
}
```

Output

```
2^3 3^2 5^1
```

This `(prime, exponent)` representation is what you'll use in most number theory problems, such as computing the number of divisors, sum of divisors, Euler's Totient Function, Möbius function, and other prime-factorization-based algorithms.